HW 2 Random Forest ML Pipeline

CSC 695 / 895

Spring 2026

Date: 3/13/2026

Name: Omeid Nadery

Email (School): onadery@sfsu.edu
Email(Personal): omeid.n@icloud.com

1. Audit of Original Database

The original dataset has 871 samples and 609 columns. One column is the label and the rest are gene expression features, so there are 608 features. There are two classes, with label 1 for the i1 neuron cluster and label 0 for non-i1. The dataset is unbalanced, with most samples in class 0, about 90%, and about 10% in class 1. All feature columns are numeric. The notebook audit checks for missing values and none were reported. No columns were flagged as potentially personally identifying, for example no columns with names such as `id`, `name`, or `patient`. The source file is `Original training DB i1 cluster.csv`.

The number of samples, 871, is below the standard of 10 times the number of features, 608, so the dataset is high-dimensional relative to sample size. Cross-validation and OOB evaluation are therefore used to assess performance without relying on a single train/test split. All statistics and results in this report use this original dataset and the train/verification split described in the next section. Parameters and settings are noted in table captions and section text.

Relevant code for loading and auditing the raw data:

```
def load_data(csv_path):
    path = Path(csv_path)
    return pd.read_csv(path)

def basic_audit(df, label_col):
    n_samples = len(df)
    feature_cols = [c for c in df.columns if c != label_col]
    n_features = len(feature_cols)
    class_counts = df[label_col].value_counts().sort_index().to_dict()
    missing_total = int(df.isna().sum().sum())
    # ... dtype check, PII heuristic, 10x features rule
    return {"n_samples": n_samples, "n_features": n_features,
            "class_counts": class_counts, ...}
```

Comment: `load_data` reads the CSV; `basic_audit` computes sample/feature counts, class distribution, missing values, and flags (e.g., high-dimensionality, imbalance) used to populate Table 1 and the narrative above.

Table 1: Original dataset statistics. Data: Original training DB i1 cluster.csv; 871 samples, 608 features, 2 classes (0: non-i1, 1: i1).

Statistic	**Value**
Number of samples	871
Number of features	608
Number of classes	2
Class 0 (non-i1)	781
Class 1 (i1)	90
Feature types	Numeric
Missing values	0
Dataset source	Original training DB i1 cluster.csv

2. Creation of Training DB and Verification DB

Two random samples were removed to form the verification set, one with class 1 and one with class 0, so the verification set has two samples, one per class. Holding out one sample per class ensures that the verification set is balanced and that the model can be checked on both i1 and non-i1 examples. The training set is the original dataset minus these two samples, so it has 869 samples. The same random seed, 42, was used so the split is reproducible.

Relevant code for the train/verification split:

```
def split_train_verification(df, label_col, random_state=RANDOM_STATE):
    pos = df[df[label_col] == POS_LABEL]
    neg = df[df[label_col] == NEG_LABEL]
    pos_sample = pos.sample(n=1, random_state=random_state)
    neg_sample = neg.sample(n=1, random_state=random_state)
    verification_df = pd.concat([pos_sample, neg_sample]).copy()
    training_df = df.drop(index=verification_df.index).copy()
    return training_df, verification_df
```

Comment: One random sample from each class is drawn with a fixed seed; the rest stays as training data. This yields the 869 vs 2 split and reproducible Tables 2 and 3.

Table 2: Training dataset statistics.

Statistic	**Value**
Number of samples	869
Number of features	608
Class 0 count	780
Class 1 count	89

Table 3: Verification dataset statistics.

Statistic	Value
Number of samples	2
Class 0 count	1
Class 1 count	1

3. Software Tools

Paths and random seed are centralized so the pipeline is reproducible:

```
# config.py
PROJECT_ROOT = Path(__file__).resolve().parent.parent
RAW_CSV_PATH = PROJECT_ROOT / "data" / "raw" / "Original training DB i1 cluster.csv"
RANDOM_STATE = 42
```

Comment: Using a single project root and fixed `RANDOM_STATE` ensures the same train/verification split and CV folds across runs.

- **Python** – Runtime and scripting.
- **Jupyter Notebook** – Interactive workflow and documentation.
- **scikit-learn** – `RandomForestClassifier` for training, `StratifiedKFold` for fold indices, and metrics including confusion matrix, accuracy, precision, recall, and F1.
- **pandas** – Loading CSV data, building tables, and saving outputs.
- **NumPy** – Numerical arrays and random state handling.
- **matplotlib / seaborn** – Top-10 feature importance plot.

4. Experimental Methods and Setup

Method 1, Random Forest with OOB estimation. Out-of-bag (OOB) estimation uses the fact that each tree in the forest is trained on a bootstrap sample of the data, so about one-third of the training samples are OOB for each tree. Those samples get predictions from the trees that did not use them, giving an internal validation estimate without a separate holdout set. A grid search was run over `n_estimators` (200, 500, 1000), `max_features` (mtry: `"sqrt"`, 2×`sqrt` ≈ 49, 5×`sqrt` ≈ 123, and 0.1 as fraction), and `min_samples_leaf` (1, 2, 5). Each model used `RandomForestClassifier` with `bootstrap=True` and `oob_score=True`. OOB predictions were taken from the out-of-bag decision function and thresholded at 0.5 (scikit-learn's fixed classification cutoff) to get predicted classes. From those, confusion matrix, accuracy, precision, recall, and F1 were computed. The best model was chosen by highest F1 because the dataset is imbalanced and has relatively few i1 samples. F1 balances precision and recall and is more informative than accuracy when the positive class is rare.

Key OOB grid and prediction logic:

```
n_features = X_train.shape[1]
sqrt_n = math.sqrt(n_features)
max_features_list = ["sqrt", min(int(2 * sqrt_n), n_features),
                     min(int(5 * sqrt_n), n_features), 0.1]
# ... loop over n_estimators, max_features, min_samples_leaf
clf = RandomForestClassifier(n_estimators=..., max_features=...,
    bootstrap=True, oob_score=True, ...)
clf.fit(X_train, y_train)
oob_probs = clf.oob_decision_function_[:, 1]
oob_pred = (oob_probs >= 0.5).astype(int)
metrics = compute_metrics(y_train, oob_pred)
```

Comment: The mtry grid is built from `sqrt` and multiples (49, 123 for 608 features); each model is evaluated via OOB class-1 probabilities thresholded at 0.5, then metrics (including F1) are computed for model selection.

Method 2, Manual 3-fold cross-validation. This method estimates how the model generalizes to unseen data by training on two-thirds of the data and evaluating on the remaining third, repeated three times so each sample is in the test set exactly once. `StratifiedKFold` with 3 splits and a fixed random state was used only to get train and test indices, without using a one-line CV shortcut. For each fold, a separate `RandomForestClassifier` was trained on the training indices and predictions were made on the test indices. Confusion matrix, accuracy, precision, recall, and F1 were computed per fold. Final accuracy and F1 are the averages over the three folds. The same hyperparameters as the best OOB model were used so that the CV results are comparable.

Key 3-fold CV loop (indices only; training and metrics are manual):

```
skf = StratifiedKFold(n_splits=3, shuffle=True, random_state=random_state)
for fold_num, (train_idx, test_idx) in enumerate(skf.split(X, y), start=1):
    X_train_fold = X.iloc[train_idx]
    X_test_fold = X.iloc[test_idx]
    y_train_fold = y.iloc[train_idx]
    y_test_fold = y.iloc[test_idx]
    clf = RandomForestClassifier(n_estimators=..., max_features=..., ...)
    clf.fit(X_train_fold, y_train_fold)
    y_pred = clf.predict(X_test_fold)
    metrics = compute_metrics(y_test_fold, y_pred)
```

Comment: `StratifiedKFold` only supplies fold indices; a separate RF is trained per fold and evaluated on that fold's test set, yielding the per-fold accuracy and F1 reported in Section 5.

5. Actual Results of RF Training

Method 1 (OOB). The best hyperparameters were `n_estimators = 1000`, `max_features = 123` (5×`sqrt`), and `min_samples_leaf = 1`.

Table 4: Best OOB confusion matrix. Full sample counts (not normalized). Rows = true class, columns = predicted class (axes: true vs. classified), order 0 then 1.

	Predicted 0	**Predicted 1**
True 0	779	1
True 1	10	79

Table 5: Best OOB performance metrics (5 decimal places). Model: n_estimators=1000, max_features=123, min_samples_leaf=1; threshold 0.5.

Metric	**Value**
Accuracy	0.98734
Precision	0.98750
Recall	0.88764
F1	0.93491

Relevant code for selecting the best OOB model and for computing metrics (used for both OOB and CV):

```
def get_best_oob_model(results, sort_by="f1"):
    best = max(results, key=lambda x: x[sort_by])
    return best

def compute_metrics(y_true, y_pred, pos_label=1):
    cm = confusion_matrix(y_true, y_pred)
    return {"confusion_matrix": cm,
            "accuracy": accuracy_score(y_true, y_pred),
            "precision": precision_score(..., pos_label=pos_label, zero_division=0),
            "recall": recall_score(..., pos_label=pos_label, zero_division=0),
            "f1": f1_score(..., pos_label=pos_label, zero_division=0)}
```

Comment: The result with highest F1 is chosen as the best OOB model. Its confusion matrix and accuracy (and precision, recall, F1) are those in the tables above. All metrics are computed via scikit-learn with the positive class set to 1 (i1).

Method 2 (3-fold CV). Per-fold results are shown below.

Table 6: Manual 3-fold CV results per fold. Same hyperparameters as best OOB model; accuracy and F1 to 5 decimal places.

Fold	**Accuracy**	**F1**
1	0.97931	0.88889
2	0.98621	0.93103
3	0.98962	0.94545

Mean accuracy was 0.98505 and mean F1 was 0.92179. Per-fold precision and recall, to 5 decimal places, are available in the notebook. Fold 1 had recall 0.80 and fold 3 had recall about 0.90, so there is some variance in how many i1 samples are correctly identified across folds.

Interpretation of the best OOB confusion matrix. The model correctly classified 779 of 780 non-i1 samples (one false positive) and 79 of 89 i1 samples (10 false negatives). So most errors are missed i1 cases rather than non-i1 samples wrongly labeled as i1. That pattern is common with imbalanced data when the minority class is harder to separate. Precision (0.98750) is higher than recall (0.88764) because the model is somewhat conservative in predicting class 1. When it does predict i1, it is usually right, but it misses some true i1 samples.

Comparison. OOB and 3-fold CV both give accuracy around 98–99% and F1 around 0.92–0.94. OOB is slightly higher (0.98734 vs. 0.98505) and the best OOB F1 (0.93491) is a bit higher than the mean CV F1 (0.92179). The two methods agree that the model performs well and is not strongly overfitting. The small gap between OOB and CV is consistent with OOB being a slightly optimistic estimate compared to evaluation on completely unseen data.

6. Feature Ranking

1. How the best trained RF was obtained. The best trained RF was obtained in Section 5 by running a grid search over `n_estimators` (200, 500, 1000), `max_features` (mtry: sqrt, 2×sqrt, 5×sqrt, 0.1), and `min_samples_leaf` (1, 2, 5), evaluating each model with OOB predictions, and selecting the configuration with highest OOB F1. That model (`n_estimators=1000`, `max_features=123`, `min_samples_leaf=1`) is the one used for feature ranking below.

2. Top 10 ranked features (Gini method for Scikit). Feature importance was computed using the **Gini method** (scikit-learn: total decrease in node impurity from splits on each feature across all trees). The top 10 ranked features are shown in Table 4 and Figure 1.

3. Code snippets.

```
def get_feature_importance_table(model, feature_names):
    importance_df = pd.DataFrame({
        "feature": feature_names,
        "gini_importance": model.feature_importances_,
    }).sort_values("gini_importance", ascending=False)
    return importance_df.reset_index(drop=True)
```

Comment: `feature_importances_` gives Gini importance per feature; we sort descending to produce the ranking in Table 4 and Figure 1.

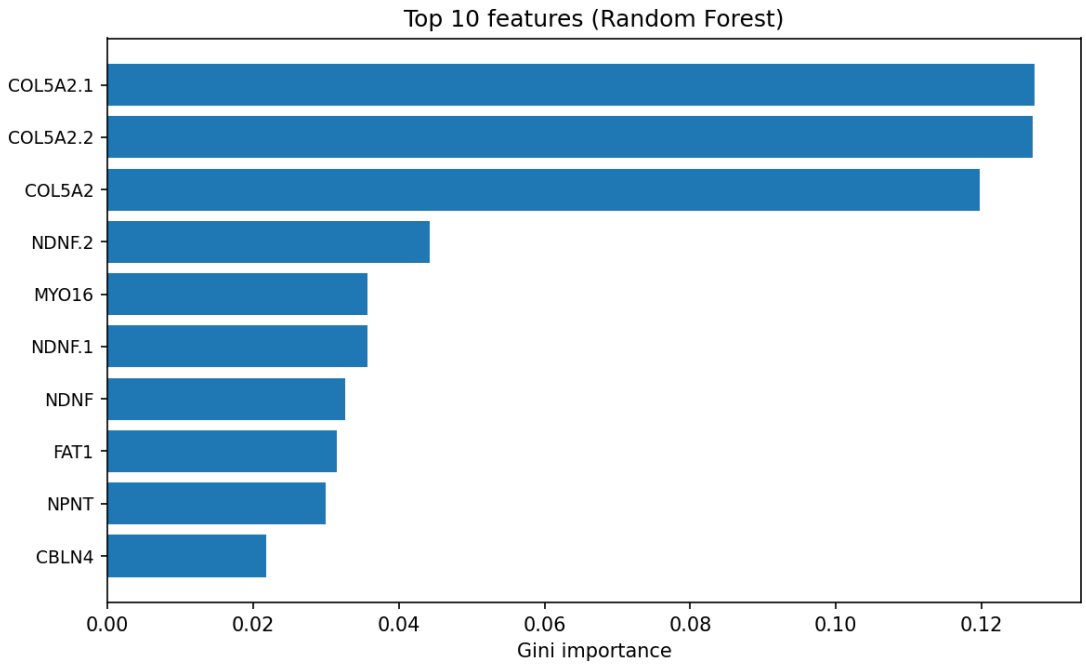

Figure 1: Top 10 features ranked by Random Forest Gini importance.

Table 7: Top 10 ranked features by Gini importance (best OOB model; measure: total decrease in node impurity across trees).

Rank	Feature	Gini importance
1	COL5A2.1	0.12725
2	COL5A2.2	0.12706
3	COL5A2	0.11970
4	NDNF.2	0.04422
5	MYO16	0.03574
6	NDNF.1	0.03569
7	NDNF	0.03266
8	FAT1	0.03151
9	NPNT	0.02999
10	CBLN4	0.02176

4. How this helps explain how the RF worked. The ranking shows which features the trees relied on most when splitting nodes, and thus which gene expression levels drove the model's i1 vs. non-i1 decisions. The COL5A2 family (top three) dominates, so the RF effectively uses collagen-related and NDNF/FAT1 expression to separate i1 from non-i1; this makes the model's behavior interpretable in terms of known biology.

5. What I learned doing this step. I learned that the learned feature importance is interpretable and recovers known i1 markers (COL5A2, NDNF, FAT1) from the literature without being given that information during training, which supports that the RF is capturing biologically

meaningful signal rather than noise. The ranking also shows a clear dominant cluster (COL5A2 family $\approx$ 37% of top-10 Gini importance), which helps justify using a small subset of features for interpretation or production.

6. Comparison with ground truth from biology paper (i1 cluster). The ground truth established in Aevermann et al. for the i1 cluster is that it is a human MTG cortical layer 1 GABAergic interneuron that selectively expresses COL5A2, NDNF, and FAT1 mRNAs. Our top 10 ranked features align with this: COL5A2 appears as the top three (COL5A2, COL5A2.1, COL5A2.2), NDNF appears in three entries (NDNF, NDNF.1, NDNF.2), and FAT1 appears once. The model's most important features correspond to the biological markers from the paper. MYO16, NPNT, and CBLN4 in the top 10 may be additional i1-related genes or correlated with the main markers.

Production suggestion: For a tradeoff between fewer features and accuracy, I would suggest using the top 5–10 ranked features in production. The top 3 alone may sacrifice some accuracy, while the full top 10 balances interpretability and performance. This should be validated on held-out data.

7. RF Runtime Test

The RF run-time engine is the best trained model from Section 5 (OOB grid), with key hyperparameters `n_estimators = 1000`, `max_features = 123`, `min_samples_leaf = 1`, and classification cutoff 0.5 (scikit-learn default). One positive (i1, class 1) and one negative (non-i1, class 0) sample from the verification database were run through this model in predict mode to obtain the predicted class and class probabilities for each sample.

Relevant code for verification predictions:

```
def run_verification_predictions(model, X_verify, y_verify):
    pred = model.predict(X_verify)
    prob = model.predict_proba(X_verify)
    results = pd.DataFrame({
        "true_class": y_verify.values,
        "predicted_class": pred,
        "prob_class_0": prob[:, 0],
        "prob_class_1": prob[:, 1],
        "correct": (pred == y_verify.values),
    })
    return results
```

Comment: The best fitted RF is run on the two held-out samples; `predict_proba` gives the probabilities reported in Table 5 and used to confirm both predictions are correct.

Table 8: Verification sample predictions. Class decision and probability for each class (prob_class_0, prob_class_1) from the run-time RF; both samples from verification database.

Sample	True label	Predicted label	P(class 0)	P(class 1)	Correct
1	1	1	0.03100	0.96900	Yes
2	0	0	0.74000	0.26000	Yes

Comparison with ground truth: Both predictions are correct. Sample 1 (true i1) is predicted as class 1. Sample 2 (true non-i1) is predicted as class 0. **Confidence in RF predictions:** My confidence is *high* for the i1 sample (probability for class 1 is 0.96900) and *moderate* for the non-i1 sample (probability for class 0 is 0.74000). I determine this from the predicted class probabilities output by the model and from the fact that both predictions match ground truth. The lower probability on the non-i1 sample suggests that sample may lie closer to the decision boundary. Overall, the runtime test shows that the best model generalizes to the held-out verification set.

8. Summary

The pipeline audited the original i1 cluster dataset (871 samples, 608 features, two classes), built a training set of 869 samples and a two-sample verification set with one sample per class, and trained Random Forest models using OOB estimation and manual 3-fold CV. The grid used mtry values `sqrt`, 2×`sqrt` (49), 5×`sqrt` (123), and 0.1. The best model (1000 trees, `max_features = 123` [5×`sqrt`], `min_samples_leaf = 1`) achieved OOB accuracy 0.98734 and F1 0.93491, with mean CV accuracy 0.98505 and mean CV F1 0.92179. Gini-based feature ranking placed the COL5A2 family, NDNF, and FAT1 at the top, matching known i1 markers from the literature. Both verification samples were classified correctly. These results support that the Random Forest pipeline is suitable for this classification task and that the learned feature importance is biologically interpretable.

9. References and Resources

Papers and tools used:

- Aevermann et al. (2018). *Cell type discovery using single-cell transcriptomics: implications for ontological representation.* Human Molecular Genetics. (Data source and biology ground truth for i1 cluster.)
- Scikit-learn documentation. https://scikit-learn.org/
- ChatGPT and Cursor (GenAI) were used for report structure and code assistance; see Appendix I.
- LateX Overleaf used for formatting this PDF https://www.overleaf.com/

A. Appendix I: GenAI Use

Tools used: ChatGPT / Cursor

Tasks assisted with:

- Helped structure the Random Forest pipeline.
- Helped review report organization and wording.
- Helped debug parts of the Python implementation.

Example prompts used:

- "How should I structure a Random Forest pipeline within a Jupyter Notebook?"
- "Give me an ideal file structure diagram for my codebase."
- "Can you review whether my metrics and feature ranking look correct?"
- "Can you give me overleaf latex code to organize my report, including tables and graphs, into a pdf."

Usefulness rating: 5/5 (Saved a lot of time formatting the file structure and report structure)

Notes: Use of AI during this homework was to help structure the project and report. ChatGPT was used to build the file structure and help prompt Cursor to help build and debug code snippets with project context. Once the results from the tests were summarized, Cursor was used for feedback on correctness of results and ChatGPT was used to format the contents of the final report into LaTeX Overleaf code for a clean PDF structure.

B. Appendix II: Full Code

HW 2 Random Forest ML pipeline for i1 cluster classification. Author: Omeid Nadery. Date: 3/13/2026. Key snippets appear in the main body; full code is below with module docstrings documenting purpose and behavior.

File: config.py

```
"""
Project configuration for HW2 Random Forest i1 cluster pipeline.
Uses pathlib for cross-platform paths (Windows/Mac/Linux).
"""
from pathlib import Path

# ---------------------------------------------------------------------------
# Project root and directories
# ---------------------------------------------------------------------------
```

```
PROJECT_ROOT = Path(__file__).resolve().parent.parent

RAW_DATA_DIR = PROJECT_ROOT / "data" / "raw"
PROCESSED_DATA_DIR = PROJECT_ROOT / "data" / "processed"
OUTPUT_DATA_DIR = PROJECT_ROOT / "data" / "outputs"
FIGURES_DIR = PROJECT_ROOT / "reports" / "figures"

RAW_CSV_PATH = RAW_DATA_DIR / "Original training DB i1 cluster.csv"
TRAIN_CSV_PATH = PROCESSED_DATA_DIR / "training_db.csv"
VERIFY_CSV_PATH = PROCESSED_DATA_DIR / "verification_db.csv"

RANDOM_STATE = 42
LABEL_COL = "label"
POS_LABEL = 1
NEG_LABEL = 0
```

File: data_utils.py

```
"""
Data loading, auditing, and train/verification split utilities.
"""
import pandas as pd
import numpy as np
from pathlib import Path
from src.config import RANDOM_STATE, POS_LABEL, NEG_LABEL

def load_data(csv_path):
    path = Path(csv_path)
    if not path.exists():
        raise FileNotFoundError(f"CSV not found: {path}")
    return pd.read_csv(path)

def detect_label_column(df):
    for c in df.columns:
        if str(c).strip().lower() == "label":
            return c
    if "label" in df.columns: return "label"
    if "Label" in df.columns: return "Label"
    raise ValueError("No label column found.")

def basic_audit(df, label_col):
    n_samples = len(df)
    feature_cols = [c for c in df.columns if c != label_col]
    n_features = len(feature_cols)
    class_counts = df[label_col].value_counts().sort_index().to_dict()
    total = n_samples
    class_pct = {k: (v / total * 100) if total else 0 for k, v in class_counts.items()}
    missing_total = int(df.isna().sum().sum())
    X = df[feature_cols]
```

```
    dtypes = X.dtypes
    all_numeric = all(pd.api.types.is_numeric_dtype(d) for d in dtypes)
    dtype_summary = {str(k): int(v) for k, v in dtypes.value_counts().to_dict().items()}
    pii_heuristic = [c for c in df.columns if any(x in str(c).lower()
        for x in ["id", "name", "patient", "sample_id", "subject"])]
    samples_ok = n_samples >= 10 * n_features if n_features else True
    min_pct = min(class_pct.values()) if class_pct else 0
    unbalanced = min_pct < 10.0
    return {"n_samples": n_samples, "n_features": n_features, "class_counts":
        class_counts,
        "class_pct": class_pct, "missing_total": missing_total, "all_features_numeric":
            all_numeric,
        "pii_heuristic_columns": pii_heuristic, "samples_at_least_10x_features":
            samples_ok,
        "class_unbalanced_under_10pct": unbalanced, "feature_columns": feature_cols}

def audit_summary_table(df, label_col):
    audit = basic_audit(df, label_col)
    rows = [("Number of samples", audit["n_samples"]), ("Number of features", audit["
        n_features"]),
        ("Class counts", str(audit["class_counts"])), ("Missing values (total)", audit["
            missing_total"]),
        ("All features numeric", audit["all_features_numeric"]),
        ("Samples >= 10x features", audit["samples_at_least_10x_features"]),
        ("Class unbalanced (<10%)", audit["class_unbalanced_under_10pct"])]
    return pd.DataFrame(rows, columns=["Audit item", "Value"])

def split_train_verification(df, label_col, random_state=RANDOM_STATE):
    pos = df[df[label_col] == POS_LABEL]
    neg = df[df[label_col] == NEG_LABEL]
    pos_sample = pos.sample(n=1, random_state=random_state)
    neg_sample = neg.sample(n=1, random_state=random_state)
    verification_df = pd.concat([pos_sample, neg_sample]).copy()
    training_df = df.drop(index=verification_df.index).copy()
    return training_df, verification_df

def get_xy(df, label_col):
    X = df.drop(columns=[label_col]).copy()
    y = df[label_col].copy()
    return X, y

def save_dataframe(df, path):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    df.to_csv(path, index=False)
```

File: rf_oob.py

```
"""
```

```python
Random Forest training with Out-of-Bag (OOB) estimation.
Grid search over hyperparameters; each model uses OOB for evaluation.
"""
import math
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from src.metrics_utils import compute_metrics

def run_oob_grid(X_train, y_train, random_state=42):
    results = []
    n_features = X_train.shape[1]
    sqrt_n = math.sqrt(n_features)
    # mtry: sqrt, 2*sqrt, 5*sqrt (capped at n_features), and 0.1 (10% fraction)
    max_features_list = [
        "sqrt",
        min(int(2 * sqrt_n), n_features),
        min(int(5 * sqrt_n), n_features),
        0.1,
    ]
    n_estimators_list = [200, 500, 1000]
    min_samples_leaf_list = [1, 2, 5]

    for n_estimators in n_estimators_list:
        for max_features in max_features_list:
            for min_samples_leaf in min_samples_leaf_list:
                clf = RandomForestClassifier(
                    n_estimators=n_estimators,
                    max_features=max_features,
                    min_samples_leaf=min_samples_leaf,
                    bootstrap=True,
                    oob_score=True,
                    random_state=random_state,
                    n_jobs=-1,
                )
                clf.fit(X_train, y_train)
                oob_probs = clf.oob_decision_function_[:, 1]
                oob_pred = (oob_probs >= 0.5).astype(int)
                metrics = compute_metrics(y_train, oob_pred)
                results.append({
                    "n_estimators": n_estimators,
                    "max_features": max_features,
                    "min_samples_leaf": min_samples_leaf,
                    "oob_score_builtin": clf.oob_score_,
                    "accuracy": metrics["accuracy"],
                    "precision": metrics["precision"],
                    "recall": metrics["recall"],
                    "f1": metrics["f1"],
                    "confusion_matrix": metrics["confusion_matrix"],
                    "model": clf,
                })
```

```
    return results

def oob_results_to_dataframe(results):
    rows = []
    for r in results:
        rows.append({"n_estimators": r["n_estimators"], "max_features": str(r["
            max_features"]),
            "min_samples_leaf": r["min_samples_leaf"], "oob_score": r["oob_score_builtin"
                ],
            "accuracy": r["accuracy"], "precision": r["precision"], "recall": r["recall"],
                 "f1": r["f1"]})
    return pd.DataFrame(rows)

def get_best_oob_model(results, sort_by="f1"):
    if not results: return None
    valid = [s for s in sort_by.split(",") if s in results[0]]
    sort_key = valid[0] if valid else "f1"
    return max(results, key=lambda x: x[sort_key])
```

File: rf_cv.py

```
"""
Manual 3-fold cross-validation for Random Forest.
Uses StratifiedKFold only for index generation; training/prediction/metrics are manual.
"""
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold
from src.metrics_utils import compute_metrics

def run_manual_3fold_cv(X, y, n_estimators, max_features, min_samples_leaf, random_state
    =42):
    skf = StratifiedKFold(n_splits=3, shuffle=True, random_state=random_state)
    fold_results = []
    for fold_num, (train_idx, test_idx) in enumerate(skf.split(X, y), start=1):
        if hasattr(X, "iloc"):
            X_train_fold, X_test_fold = X.iloc[train_idx], X.iloc[test_idx]
            y_train_fold, y_test_fold = y.iloc[train_idx], y.iloc[test_idx]
        else:
            X_train_fold, X_test_fold = X[train_idx], X[test_idx]
            y_train_fold, y_test_fold = y[train_idx], y[test_idx]
        clf = RandomForestClassifier(n_estimators=n_estimators, max_features=max_features,
            min_samples_leaf=min_samples_leaf, random_state=random_state, n_jobs=-1)
        clf.fit(X_train_fold, y_train_fold)
        y_pred = clf.predict(X_test_fold)
        metrics = compute_metrics(y_test_fold, y_pred)
        fold_results.append({"fold": fold_num,
```

```
            "train_class_0": int((y_train_fold == 0).sum()), "train_class_1": int((
                y_train_fold == 1).sum()),
            "test_class_0": int((y_test_fold == 0).sum()), "test_class_1": int((
                y_test_fold == 1).sum()),
            "accuracy": metrics["accuracy"], "precision": metrics["precision"],
            "recall": metrics["recall"], "f1": metrics["f1"], "confusion_matrix": metrics[
                "confusion_matrix"]})
    return fold_results

def cv_results_to_dataframe(results):
    return pd.DataFrame([{"fold": r["fold"], "train_class_0": r["train_class_0"], "
        train_class_1": r["train_class_1"],
        "test_class_0": r["test_class_0"], "test_class_1": r["test_class_1"],
        "accuracy": r["accuracy"], "precision": r["precision"], "recall": r["recall"], "f1
            ": r["f1"]} for r in results])

def summarize_cv_results(results):
    df = cv_results_to_dataframe(results)
    return {"mean_accuracy": df["accuracy"].mean(), "mean_precision": df["precision"].
        mean(),
        "mean_recall": df["recall"].mean(), "mean_f1": df["f1"].mean(),
        "std_accuracy": df["accuracy"].std(), "std_f1": df["f1"].std()}
```

File: feature_rank.py

```
"""
Feature importance ranking from a trained Random Forest (Gini importance).
"""
import pandas as pd

def get_feature_importance_table(model, feature_names):
    importance_df = pd.DataFrame({
        "feature": feature_names,
        "gini_importance": model.feature_importances_,
    }).sort_values("gini_importance", ascending=False)
    return importance_df.reset_index(drop=True)

GROUND_TRUTH_GENES = ["COL5A2", "NDNF", "FAT1"]
COL5A2_PREFIX = "COL5A2"

def summarize_ground_truth_overlap(feature_df, top_n=10):
    top = feature_df.head(top_n)
    features_list = top["feature"].astype(str).tolist()
    matches = {"COL5A2_family": [], "NDNF": [], "FAT1": []}
    for f in features_list:
        f_upper = f.upper()
        if f_upper.startswith(COL5A2_PREFIX): matches["COL5A2_family"].append(f)
        if "NDNF" in f_upper: matches["NDNF"].append(f)
        if "FAT1" in f_upper: matches["FAT1"].append(f)
```

```
    summary_parts = []
    if matches["COL5A2_family"]: summary_parts.append("COL5A2 family: " + ", ".join(
        matches["COL5A2_family"]))
    if matches["NDNF"]: summary_parts.append("NDNF: " + ", ".join(matches["NDNF"]))
    if matches["FAT1"]: summary_parts.append("FAT1: " + ", ".join(matches["FAT1"]))
    if not summary_parts: summary_parts.append("None of COL5A2, NDNF, FAT1 in top-ranked
        features.")
    return {"matches": matches, "summary": "; ".join(summary_parts), "top_features":
        features_list}
```

File: runtime_test.py

```
"""
Run the best trained model on the 2 held-out verification samples.
"""
import pandas as pd

def run_verification_predictions(model, X_verify, y_verify):
    pred = model.predict(X_verify)
    prob = model.predict_proba(X_verify)
    return pd.DataFrame({
        "true_class": y_verify.values,
        "predicted_class": pred,
        "prob_class_0": prob[:, 0],
        "prob_class_1": prob[:, 1],
        "correct": (pred == y_verify.values),
    })
```

File: metrics_utils.py

```
"""
Metrics computation and formatting for classification.
"""
import pandas as pd
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score,
    recall_score, f1_score

DEFAULT_POS_LABEL = 1

def compute_metrics(y_true, y_pred, pos_label=DEFAULT_POS_LABEL):
    cm = confusion_matrix(y_true, y_pred)
    return {"confusion_matrix": cm,
        "accuracy": accuracy_score(y_true, y_pred),
        "precision": precision_score(y_true, y_pred, pos_label=pos_label, zero_division=0)
            ,
        "recall": recall_score(y_true, y_pred, pos_label=pos_label, zero_division=0),
        "f1": f1_score(y_true, y_pred, pos_label=pos_label, zero_division=0)}
```

```python
def format_metric(value, decimals=5):
    if isinstance(value, (int, float)):
        return f"{value:.{decimals}f}"
    return str(value)

def confusion_matrix_to_frame(cm, index_labels=None, columns_labels=None):
    if index_labels is None: index_labels = ["0", "1"]
    if columns_labels is None: columns_labels = ["0", "1"]
    return pd.DataFrame(cm, index=index_labels, columns=columns_labels)
```

File: report_helpers.py

```python
"""
Helpers for saving result tables to CSV and figures.
"""
import pandas as pd
from pathlib import Path
from src.config import OUTPUT_DATA_DIR, FIGURES_DIR
from src.metrics_utils import format_metric

def save_result_table_to_csv(df, filename, output_dir=None):
    out_dir = output_dir or OUTPUT_DATA_DIR
    out_dir = Path(out_dir)
    out_dir.mkdir(parents=True, exist_ok=True)
    path = out_dir / filename
    df.to_csv(path, index=False)
    return path

def save_top10_feature_plot(feature_df, filepath=None):
    import matplotlib.pyplot as plt
    top10 = feature_df.head(10)
    fig, ax = plt.subplots(figsize=(8, 5))
    ax.barh(range(len(top10)), top10["gini_importance"].values, align="center")
    ax.set_yticks(range(len(top10)))
    ax.set_yticklabels(top10["feature"].tolist(), fontsize=9)
    ax.invert_yaxis()
    ax.set_xlabel("Gini importance")
    ax.set_title("Top 10 features (Random Forest)")
    plt.tight_layout()
    if filepath is None:
        FIGURES_DIR.mkdir(parents=True, exist_ok=True)
        filepath = FIGURES_DIR / "top10_features.png"
    else:
        filepath = Path(filepath)
    filepath.parent.mkdir(parents=True, exist_ok=True)
    plt.savefig(filepath, dpi=150, bbox_inches="tight")
    plt.close()
    return filepath
```